Algorithms & Data Structures

Time & Space Complexity

Basics

Best Case (Ω)- the minimum time an algorithm takes on the best possible input. Represents the lower bound of performance.

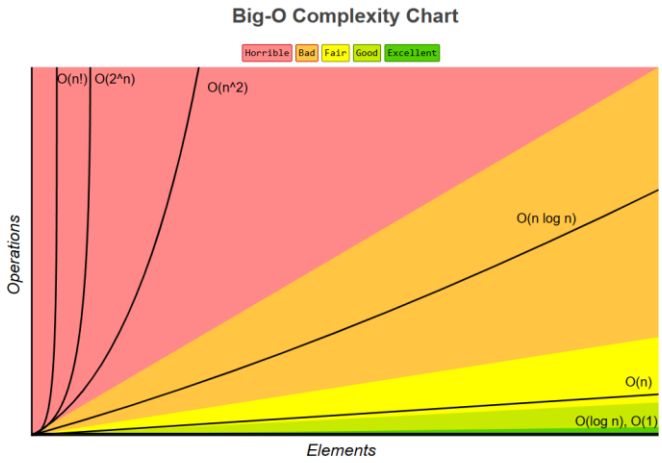
Example: Ω(1)- Finding an element at the first position using linear search.

Average Case (Θ)- the expected time an algorithm takes across all inputs. Represents the tight bound (both upper and lower) under typical conditions.

Example: Θ(n) - Linear search where the target is randomly located.

Worst Case (O)- the maximum time an algorithm takes on the worst possible input. Represents the upper bound of performance.

Example: O(n) - Searching for a non-existent element in an unsorted array.



Amortized Time - it's like we use average time complexity, and ignore worst case complexity because worst case happens in rare case scenarios.

Example: If we add new element in an array in some cases that addition can resize array which takes O(n), but that resize occurs in rare cases and we can ignore it and say insertion is O(1).

Interview Questions

1. What is best, average and worst complexity?
2. What is Amortized complexity?
3. When is okay to use higher time complexity instead of lower time?
4. Example of an algorithm where the best-case and worst-case complexities are different.

Exercises

Calculate Time and Space complexity:

1.

```cpp
void printAllPairs(const std::vector<int>& vec) {
    int n = vec.size();
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            std::cout << vec[i] << ", " << vec[j] << std::endl;
        }
    }
}
```

2.

```cpp
long long process(std::vector<int> data) {
    int n = data.size();
    if (n <= 1) {
        return 1;
    }

    long long work_counter = 0;
    for(int i = 0; i < n; ++i) {
        work_counter++;
    }

    std::vector<int> left_half(data.begin(), data.begin() + n / 2);
    std::vector<int> right_half(data.begin() + n / 2, data.end());

    return work_counter + process(left_half) + process(right_half);
}
```

3.

```cpp
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

4.

```cpp
// Counts the frequency of each number and processes queries.
void countAndQuery(const std::vector<int>& data, const std::vector<int>& queries) {
    // NOTE: std::map is a balanced binary search tree (Red-Black Tree).
    std::map<int, int> counts;

    // Populate the map
    for (int val : data) {
        counts[val]++;
    }

    // Process the queries
    for (int q : queries) {
        if (counts.count(q)) {
            std::cout << "Query " << q << " found." << std::endl;
        }
    }
}
```

5.

```cpp
void geometricSeriesWork(int n) {
    long long total_operations = 0;
    for (int i = n; i > 0; i /= 2) {
        for (int j = 0; j < i; ++j) {
            total_operations++;
        }
    }
    std::cout << "Total operations: " << total_operations << std::endl;
}
```

Arrays

Static vs Dynamic Array

In statically-typed languages all elements in an array must be of the same data type. This allows the computer to calculate the exact memory location of any element easily.

`memory_address = base_address + (index * size_of_one_element)`

In dynamically-typed languages like a single list/array can hold elements of different types, but "under the hood" it's **often an array of pointers** to the actual objects, which are stored elsewhere.

Usage

- When we know the number of elements in advance.
- When we need very fast access to elements by their position (index).

- When we need to store a sequence of data and will mostly be iterating over it or accessing elements directly.
- As the underlying structure for other data structures, like stacks, queues, and hash tables.

Stringstream

When we use += to append to a std::string, we are potentially performing a very expensive operation.The string checks if its current capacity is large enough to hold the new characters. If the capacity is exceeded, the string must allocate a larger block of memory, copy all the characters from the old memory block to the new one, and append the new content to the end of the new block.

Stringstream uses an internal buffer, optimized for many small appends. It grows its buffer in larger, more intelligent chunks, leading to far fewer re-allocations than the += loop. All the appends happen in this efficient in-memory buffer.

Because the capacity grows geometrically, the expensive O(n) re-allocations become increasingly infrequent as the string gets larger. This leads to Amortized O(1) time.

Interview Questions

1. Why is iterating through an array often faster than iterating through a linked list of the same size?

Commented [VJ1]: *Arrays have better spatial locality, meaning they fit into the CPU cache better because elements are contiguous in memory.*

Linked Lists

Basics

Linked List organizes data in individual objects called **Nodes** (instead of contiguous blocks). Each node contains **data and a pointer** to the next node.

Head - a pointer that always points to the first node in the list. If the head is null, the list is empty. **Losing the head is like losing the entire list**, because you have no way to find the first node.

Usage

- When we have more insert/delete operations.
- If we insert and delete often (arrays can make disk fragmentation).

Feature	Array	Linked List
Memory Layout	**Contiguous** (all elements are side-by-side in memory).	**Non-contiguous** (nodes can be scattered anywhere in memory).
Size	**Static** (fixed size) or requires resizing (costly).	**Dynamic** (can grow and shrink easily one node at a time).
Access (Read)	**O(1)** - Constant time. `array[i]` is a direct calculation.	**O(n)** - Linear time. Must traverse from the `head` to find the i-th element.
Insertion/Deletion	**O(n)** - Linear time. May need to shift all subsequent elements.	**O(1)** - Constant time (if you have a pointer to the node *before* the target).
Insertion/Deletion (at start)	**O(n)** - Must shift every element.	**O(1)** - Just update the `head` pointer.
Insertion/Deletion (at end)	**O(1)** (if not resizing).	**O(n)** (if no `tail` pointer) or **O(1)** (if `tail` pointer is maintained).

Problem Solving Patterns

1. Fast & Slow Pointers - one pointer (slow) moves one step at a time, while the other (fast) moves two steps at a time.
2. Dummy/Sentinel Head Node - placeholder node that we add before the actual head of list.
3. Multiple Pointers - previous, current and next pointer, or current and next.
4. Two Pointers at a Fixed Distance - one pointer moves N steps, and then both pointers move by a single step.

Interview Questions

1. Implement Double Linked List
2. Implement Cyclic Linked List
3. Reverse Linked List
4. Linked List Cycle
5. Remove Nth Node From End of List
6. Remove Linked List Elements
7. Convert Binary Number in a Linked List to Integer
8. Swap Nodes in Pairs
9. Odd Even Linked List
10. Partition List
11. Rotate List
12. Merge in Between Linked Lists
13. Split a Circular Linked List
14. Insert Greatest Common Divisors in Linked List
15. Delete Nodes From Linked List Present in Array
16. Find the Minimum and Maximum Number of Nodes Between Critical Points
17. Palindrome Linked List

18. (⋆) Reverse Nodes in k-Group
19. (⋆⋆⋆) Implement Skip List

Stack

Basics

Stack is a linear data structure that follows the **Last-In, First-Out (LIFO)** principle. The last element added to the stack is the first one to be removed.

The primary operations on a stack are:

- **push()**: Adds an element to the top of the stack.
- **pop()**: Removes the top element from the stack.
- **peek()** or **top()**: Returns the top element without removing it.
- **isEmpty()**: Checks if the stack is empty.

Usage

- Programming languages use a call stack to manage function calls.
- In software like text editors, actions are pushed onto a stack to be easily undone.
- Stacks are used to evaluate arithmetic expressions, particularly in postfix or prefix notation.
- Stacks help in checking for balanced parentheses in code.
- Used in Depth-First Search (DFS) for traversing graphs or trees.

Problem Solving Patterns

1. **Monotonic Stack** - used to find the next greater or smaller element in an array. Maintains a sequence of elements in either increasing or decreasing order.
2. **Balancing Symbols** - validate expressions with balanced symbols.
3. **Expression Evaluation** - evaluate Reverse Polish Notation (RPN) or to convert infix expressions to postfix.
4. **Simulation of Recursion** - any recursive function can be implemented iteratively using a stack to simulate the function call stack.

Interview Questions

1. Valid Parentheses
2. Backspace String Compare
3. Next Greater Element I
4. Next Greater Element II

5. Min Stack
6. Evaluate Reverse Polish Notation
7. Daily Temperatures
8. Asteroid Collision
9. Decode String
10. (★) Largest Rectangle in Histogram

Queue

Basics

Queue is a linear data structure that operates on a **First-In, First-Out (FIFO)** basis. It is similar to a real-world queue or line, where the first person to join is the first person to be served.

The primary operations for a queue are:

- **enqueue()** or **push()**: Adds an element to the rear of the queue.
- **dequeue()** or **pop()**: Removes an element from the front of the queue.
- **front()** or **peek()**: Returns the front element without removing it.
- **isEmpty()**: Checks if the queue is empty.

Circular Queue is a linear data structure that follows the FIFO principle, but its last position is connected back to the first, creating a circle. It's typically implemented with a fixed-size array.

Deque is a generalization of a queue that allows adding and removing elements from both the front and the back. It can function as both a queue (FIFO) and a stack (LIFO).

Usage

- Operating systems use queues to manage processes waiting for the CPU.
- For traversing a graph or tree level by level.
- In networking, queues are used to handle data packets.
- Managing print jobs in the order they are submitted.

Problem Solving Patterns

1. **Monotonic Queue** - a queue where elements are kept in a strictly increasing or decreasing order. When adding a new element, remove elements from the back that break the order.
2. **Rate Limiting / Moving Average** - store timestamps or values within a specific time window. As time progresses, old timestamps are dequeued from the front if they fall out of the window.

3. **Sliding Window with Deque** - efficiently add/remove elements from both ends to track specific properties within the current window.

Interview Questions

1. Implement Queue using Stacks
2. Implement Stack using Queues
3. Number of Recent Calls
4. Time Needed to Buy Tickets
5. Design Circular Queue
6. Reveal Cards In Increasing Order
7. Dota2 Senate
8. (**) Sliding Window Maximum
9. (***) Shortest Subarray with Sum at Least K

Binary Trees

Basics

Binary Tree is a hierarchical data structure in which each node has at most two children, referred to as the **left child** and the **right child**.

- **Root:** The topmost node in the tree.
- **Parent/Child:** A node is a parent to the nodes it connects to below it.
- **Leaf:** A node that has no children.
- **Height:** The number of edges on the longest path from the root to a leaf.
- **Depth:** The number of edges from the root to a specific node.

Types of Binary Trees

1. **Full Binary Tree** - every node has either 0 or 2 children.
2. **Complete Binary Tree** - all levels are completely filled except possibly the last, which is filled from left to right.
3. **Perfect Binary Tree** - all internal nodes have two children and all leaves are at the same level.
4. **Balanced Binary Tree** - the height of the left and right subtrees of every node differs by no more than 1.

Binary Tree Traversals

- **In-order (Left, Root, Right)** - used in Binary Search Trees to get elements in non-decreasing order.
- **Pre-order (Root, Left, Right)** - used to create a copy of the tree or for prefix expression notation. Best for Top-down problems where the parent passes information down.
- **Post-order (Left, Right, Root)** - used to delete the tree or for postfix expression notation. Best for Bottom-up problems.
- **Level-order** - visiting nodes level by level (Breadth-First Search).

Usage

- Used for efficient searching, insertion, and deletion (BST).
- Used by compilers to evaluate mathematical expressions.
- Used in data compression algorithms.
- Used in parsing programming languages.

Problem Solving Patterns

1. **Recursive Depth-First Search (DFS)** - most tree problems can be solved by breaking them into sub-problems: solve(root->left) and solve(root->right).
   1a. Top-Down- *What do I need to tell my children so they can solve the problem?*
   1b. Bottom-Up- *What information do my children have that I need to combine?*
2. **Level Order Traversal (BFS)** - process the tree layer by layer. Useful for finding the shortest path or minimum depth.
3. **Lowest Common Ancestor (LCA)** - finding the lowest node in a tree that has two given nodes as descendants.
4. **Tree Construction** - rebuilding a tree using two traversal arrays (Pre-order and In-order).
5. **Morris Traversal** - a method to traverse the tree in O(1) space without using recursion or a stack. It works by temporarily pointing the predecessor's right pointer to the current node to create a way back up the tree.

Recursion Tricks

- **Return a Node/Value:** Use when the parent needs the actual result from the child to make a decision.
- **Return a Boolean:** Use when the problem asks "Is it...?" or "Does it have...?".
- **Return an Integer:** Use when need to calculate something based on the size or distance.
- **Void + Global/Reference:** Use when need to track a "Global Best" while doing something else.

Interview Questions

1. Binary Tree Inorder Traversal
2. Maximum Depth of Binary Tree
3. Invert Binary Tree
4. Same Tree
5. Symmetric Tree
6. Diameter of Binary Tree
7. Subtree of Another Tree
8. Binary Tree Level Order Traversal
9. Lowest Common Ancestor of a Binary Tree
10. Validate Binary Search Tree
11. Path Sum II
12. Binary Tree Right Side View
13. Populating Next Right Pointers in Each Node
14. Kth Smallest Element in a BST
15. Convert Sorted Array to Binary Search Tree
16. Trim a Binary Search Tree
17. Construct Binary Tree from Preorder and Inorder Traversal
18. Flatten Binary Tree to Linked List
19. Balance a Binary Search Tree
20. (★) Serialize and Deserialize Binary Tree
21. (★) Binary Tree Maximum Path Sum
22. (★★) Binary Tree Cameras
23. (★) Vertical Order Traversal of a Binary Tree
24. (★) All Nodes Distance K in Binary Tree

Self-Balancing Binary Search Trees

Basics

A standard BST can become skewed (like a linked list) if elements are inserted in sorted order, leading to O(n) time complexity. Self-Balancing Trees **automatically adjust structure during insertion and deletion** to keep the height at **O(log n)**.

- **Balance Factor:** The difference between the height of the left and right subtrees.
- **Rotations:** The primary mechanism used to rebalance a tree.
    - **Left Rotation**- used when a node is right-heavy.
    - **Right Rotation**- used when a node is left-heavy.

Types of Self-Balancing Binary Search Trees

- **AVL Trees:** Strictly balanced. The balance factor of every node must be -1, 0, or 1.
- **Red-Black Trees**: Less strictly balanced than AVL, but requires fewer rotations during insertions/deletions (used in C++ std::map).

Usage

- Used to implement std::map, std::set in C++ and TreeMap/TreeSet in Java.
- Used in kernel-level memory allocation (like the Linux Virtual Memory System) to track free memory blocks.

Interview Questions

1. When should you use an AVL tree over a Red-Black tree?
2. Simulate AVL Tree Insertion/Deletion on Whiteboard
3. (★★★) Implement AVL Tree
4. (★★★) Implement Red-Black Tree

Commented [VJ2]: *Use an AVL tree when the application involves frequent lookups and infrequent updates, because AVL trees are more strictly balanced and provide faster search times.*

B-Trees

Basics

B-Tree is a self-balancing search tree that allows nodes to have more than two children. It is optimized for systems that read and write large blocks of data.

- **Order (M):** The maximum number of children a node can have.
- **Keys**: Each node contains multiple sorted keys that act as separation points for its children.
- **B+ Tree**: A variant where all data is stored in the leaf nodes, and the internal nodes only store keys for navigation. (MySQL and PostgreSQL use this).

Usage

- Almost all relational databases (SQL) use B+ Trees to find rows quickly.
- NTFS (Windows), HFS+ (Mac), and ext4 (Linux) use B-Trees to manage file locations on hard drive.

Interview Questions

1. Why do databases use B+ Trees instead of Hash Maps?
2. What is the difference between a B-Tree and a B+ Tree?
3. Why not use an AVL tree for a database index?

Commented [VJ3]: *Hash maps are O(1) for single lookups, but they cannot perform range queries. B+ Trees keep data sorted, allowing for efficient range scans.*

Commented [VJ4]: *In a B-Tree, data can stay in internal nodes. In a B+ Tree, all data is in the leaves, and the leaves are linked together in a Linked List, making it even faster to scan through data in order.*

Commented [VJ5]: *The height would be too tall, requiring too many slow disk seeks.*

N-ary Trees

Basics

N-ary Tree is a tree that allows a node to have at most N children. While a Binary Tree is limited to two, an N-ary tree uses a collection (usually a list or vector) to store pointers to its children.

Traversals:

- **Pre-order:** Visit the root, then visit each child from left to right.
- **Post-order:** Visit all children first, then visit the root.

Note: In-order traversal is not standard for N-ary trees because it is ambiguous where the root should be placed between N children.

Usage

- HTML tags are structured as an N-ary tree (a can have many children).
- A folder (root) can contain any number of files or sub-folders (children).

Problem Solving Patterns

1. **The For-Loop Recursion** - in binary trees, we call dfs(left) and dfs(right). In N-ary trees, we must use a loop.
2. **Level Order (BFS)** - still uses a Queue, but when dequeue a node, must push all of its children into the queue using a loop.
3. **Left-Child Right-Sibling (LCRS)** - a pattern used to represent an N-ary tree using a binary tree structure to save memory.

Interview Questions

1. N-ary Tree Preorder Traversal
2. N-ary Tree Postorder Traversal
3. Maximum Depth of N-ary Tree
4. N-ary Tree Level Order Traversal
5. Find Root of N-ary Tree
6. Construct Quad Trees
7. (★★★) Encode N-ary Tree to Binary Tree

Heaps

Basics

Heap is a specialized **Complete Binary Tree** that satisfies the Heap Property. It is the most efficient way to access the highest or lowest priority element.

- **Min-Heap**: The value of the root is the minimum among all nodes. Every parent is smaller than its children.
- **Max-Heap**: The value of the root is the maximum among all nodes. Every parent is larger than its children.

Because it is a complete tree, we don't need pointers. We store it in an array where for any element at index i:

Left Child	**2i + 1**
Right Child	**2i + 2**
Parent	**(i- 1) / 2**

Heapify - an algorithm to build a heap optimally from an unsorted array in **O(n)**. It shifts down nodes starting from the last non-leaf node up to the root. It's more efficient than inserting elements one by one, which takes **O(n logn)** time.

Usage

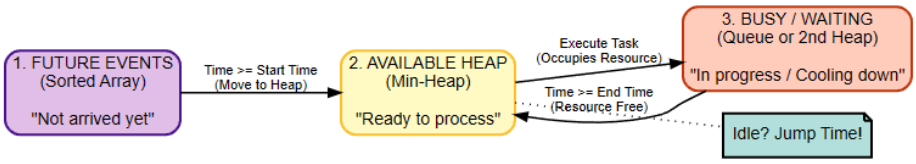
- **Priority Queues** - handling tasks based on priority rather than arrival time.
- **Heapsort** - a sorting algorithm with **O(n logn)** time and **O(1)** space.
- When need to constantly track the top or middle elements of incoming data in stream.

Problem Solving Patterns

1. **Top K Elements** - use a Min-Heap of size K to track the largest elements. If the next element is larger than the heap's root, replace it.
2. **Two Heaps Pattern** - use two heaps (usually a Max-Heap and a Min-Heap) to divide data into two parts. This allows O(1) access to the extremes of both halves.
3. **K-way Merge** - use a heap to merge K sorted arrays. The heap stores the smallest current element from each array.
4. **Task Scheduling** - use a heap to track which task has the earliest deadline or highest priority.
5. **Heap + Queue** - used when processing elements based on priority, but processed elements must wait before re-entering the pool.

Event-Driven Simulation

- **Three-Container Strategy** - organize data into three states:
  - **Input Array** - sort by arrival time (these are tasks that haven't started yet).
  - **The Heap** - stores only tasks that are currently available. Sort by the problem's priority.
  - **Current Time Variable** - a pointer t tracking the simulation clock (if the Heap is empty, jump t directly to the arrival time of the next task in the input).



Interview Questions

1. Kth Largest Element in a Stream
2. Last Stone Weight
3. Relative Ranks
4. The K Weakest Rows in a Matrix
5. Largest Number After Digit Swaps by Parity
6. Kth Largest Element in an Array
7. K Closest Points to Origin
8. Top K Frequent Elements
9. Single-Threaded CPU
10. Car Pooling
11. Task Scheduler
12. Reorganize String
13. The Number of the Smallest Unoccupied Chair
14. (*) Merge K Sorted Lists
15. (*) Find Median from Data Stream
16. (*) Smallest Range Covering Elements from K Lists
17. (*) IPO

Tries

Basics

Trie is a tree-based data structure used to efficiently store and retrieve keys in a dataset of strings. Nodes in a Trie do not store the key associated with that node. Instead, the position of the node in the tree defines the key.

The root is empty. Each node contains an array of pointers (usually size 26 for English lowercase) or a Hash Map to its children.

Each node usually has a boolean flag to indicate if the path from the root to this node forms a valid word.

Usage

- **Autocomplete** - when a user types a word, the Trie can quickly find all words starting with that prefix.
- **Spell check** - validating if a word exists in a dictionary.
- **IP Routing** - longest Prefix Match algorithms in routers.

Problem Solving Patterns

1. **The Augmented Trie** - don't just store isEnd. Store aggregate data about the subtree at every node. Store data such as: how many words use character, the maximum weight of any word in this branch, a list of indices of all words passing through node.
2. **Compound Keys (Wrapping)** - if we need Suffix or Middle search, transform the data before insertion. Insert words in reverse or wrap the string (to search for start='a', end='b', insert {b}#{a}).
3. **Trie Pruning (Dynamic Backtracking)** - trie shrinks as we find answers. For example: when DFS finds a word in the grid, it marks it as found and immediately removes it from the Trie.
4. **Bitwise Trie** - storing binary representations of numbers (0 goes left, 1 goes right) to solve Maximum XOR problems.
5. **Trie Serialization (Merkle Trees)** - used when the shape of the Trie matters, not just the path.
    1. Perform a Post-Order Traversal.
    2. For every node, construct a string signature **(child1_sign) + (child2_sign).**
    3. Hash signature. If two nodes have the same hash, their entire subtrees are identical.

Interview Questions

1. Implement Trie
2. Design Add and Search Words Data Structure
3. Longest Word in Dictionary
4. Replace Words
5. Implement Magic Dictionary
6. Map Sum Pairs
7. Camelcase Matching
8. Search Suggestions System
9. (★★) Word Search II
10. (★★) Stream of Characters
11. (★★) Prefix and Suffix Search
12. (★★★) Delete Duplicate Folders in System

Hash Maps

Basics

Hash Map is a data structure that implements an associative array abstract data type, a structure that can map keys to values. It uses a hash function to compute an index into an array of buckets or slots.

- **Hash Function**: Converts a key (string, integer, object) into an integer index. It must be deterministic.
- **Collisions**: When two keys hash to the same index.
    - **Chaining**: The bucket contains a Linked List (or Balanced BST) of all items at that index.
    - **Open Addressing**: Finds the next open slot in the array (Linear/Quadratic Probing).
- **Load Factor**: The ratio of the number of stored elements to the total number of buckets. It measures how full the hash map is getting.
- **Rehashing:** When the load factor exceeds a specific threshold, the hash map automatically resizes. It then recalculates the hash for all existing elements to redistribute them into the new buckets. It takes **O(n)** time, but keeps the average lookup at amortized **O(1)**.

Note: In C++, std::map is NOT a Hash Map. It is a Red-Black Tree. Its operations are O(logn). std::unordered_map is the actual Hash Map.

Prefix Sums

Prefix Sum algorithm uses an array to store **running totals**, allowing O(1) range sum queries. When combined with a Hash Map, this technique allows us to solve Subarray Sum problems in O(n).

The sum of a subarray from index i to j is the difference between their prefix sums.

$$\boxed{\boldsymbol{Sum(i,j) = PrefixSum[j] - PrefixSum[i-1]}}$$

Usage

- **Databases** - indexing data for fast retrieval.
- **Caching** - storing computational results to avoid re-calculation (**Memoization**).
- **Counting** - calculating frequencies of elements.

Problem Solving Patterns

1. **Frequency Map / Counter** - store element -> count. Useful for anagrams, majority elements, or finding uniques.
2. **Two Sum Pattern** - when looking for a pair that sums to a target, iterate and check if target - current_value exists in the map. Stores the complement we need.
3. **Prefix Sum + Hash Map** - store cumulative_sum -> index. Used to find subarrays that sum to K. If curr_sum - K exists in the map, a subarray ending here with sum K exists.
4. **Map + Binary Search** - store a sorted list for value (map<key, vector<int>>) and use binary search on the vector to find value.
5. **Linked Hash Map** - combines a Hash Map with a Doubly Linked List.

Interview Questions

1. Two Sum
2. Valid Anagram
3. Intersection of Two Arrays
4. First Unique Character in a String
5. Group Anagrams
6. Contiguous Array
7. Subarray Sum Equals K
8. Product of Array Except Self
9. Valid Sudoku
10. Brick Wall
11. Repeated DNA Sequences
12. Insert Delete GetRandom O(1)

13. (★) Design HashMap
14. (★) Snapshot Array
15. (★) Time Based Key-Value Store
16. (★) LRU Cache
17. (★★★) LFU Cache

Sets

Basics

Set is a collection of unique elements. Unlike arrays, sets do not allow duplicate values. They are primarily used to check for the existence of an element efficiently.

- **Unordered Set (std::unordered_set)** - O(1) average for insert, delete, and search. Elements are stored in no particular order.
- **Ordered Set (std::set)** - O(logn) for insert, delete, and search. Elements are always sorted (ascending by default).

Usage

- **Deduplication** - converting a list with duplicates into a set to removes them.
- **Fast Lookups** - checking if an element exists in O(1).
- **Graph Traversal** - keeping track of visited nodes in BFS/DFS to prevent cycles.

Problem Solving Patterns

1. **The "Seen" Set** - iterate through a collection and store elements in a set. Before adding, check if element is seen. If it does, we found a duplicate.

Interview Questions

1. Contains Duplicate
2. Happy Number
3. Find All Numbers Disappeared in an Array
4. Distribute Candies
5. Longest Consecutive Sequence

Sorting Algorithms

Basics

Sorting is the process of arranging data in a specific order (increasing or decreasing).

A sorting algorithm is **stable** if two objects with equal keys appear in the same order in the sorted output as they appear in the input.

If we sort a list of students by name, then by grade, a stable sort preserves the Name order for students with the same Grade.

Algorithms

Elementary Sorts O(N^2)

Bubble Sort [Stable] - repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The largest elements bubble to the top. Rarely used in real world.
Insertion Sort [Stable] - builds the final sorted array one item at a time. It takes an element and shifts previous elements up to find the correct insertion spot (like sorting playing cards in hand). Minimizes writes, but generally slow.
Selection Sort [Unstable] - divides the list into a sorted and unsorted part. Repeatedly finds the minimum element from the unsorted part and swaps it with the first unsorted element. Used by standard libraries for small subarrays.

Efficient Sorts O(N log N)

Quick Sort [Unstable] - a Divide and Conquer algorithm. It picks a pivot element and partitions the array so that all elements smaller than the pivot are on the left, and larger ones are on the right. It then recursively sorts the sub-arrays. Preferred for arrays due to cache efficiency.
Merge Sort [Stable] - recursively divides the array into two halves until they are size 1, then merges the sorted halves back together. Preferred for linked lists and external sorting (disk).
TimSort [Stable] - a hybrid of Merge Sort and Insertion Sort. It identifies natural runs of sorted data, sorts small runs with Insertion Sort, and merges them using Merge Sort logic. Optimized for real-world data.
Heap Sort [Unstable] - converts the array into a Heap. It repeatedly extracts the maximum element and moves it to the end of the array.

Linear Sorts O(N)

Counting Sort [Stable] - creates a frequency array (count of each value). Reconstructs the sorted array using these counts. Only efficient if range K is small (ages, alphabet).
Radix Sort [Stable] - sorts numbers digit by digit (from least significant to most significant) using a stable subroutine (Counting Sort). Used for sorting integers or strings with fixed length.
Bucket Sort [Stable] - distributes elements into several buckets (usually based on range). Each bucket is then sorted individually (using another sort), and buckets are concatenated. Useful for uniformly distributed data.

Pattern-Specific Sorts

Cyclic Sort [Unstable] - iterates through an array containing numbers range 1 to N. Swaps element nums[i] to its correct index nums[i]- 1. Only works for specific find missing/duplicate number problems.

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	Ω(n log(n))	θ(n log(n))	O(n^2)	O(log(n))
Mergesort	Ω(n log(n))	θ(n log(n))	O(n log(n))	O(n)
Timsort	Ω(n)	θ(n log(n))	O(n log(n))	O(n)
Heapsort	Ω(n log(n))	θ(n log(n))	O(n log(n))	O(1)
Bubble Sort	Ω(n)	θ(n^2)	O(n^2)	O(1)
Insertion Sort	Ω(n)	θ(n^2)	O(n^2)	O(1)
Selection Sort	Ω(n^2)	θ(n^2)	O(n^2)	O(1)
Tree Sort	Ω(n log(n))	θ(n log(n))	O(n^2)	O(n)
Shell Sort	Ω(n log(n))	θ(n(log(n))^2)	O(n(log(n))^2)	O(1)
Bucket Sort	Ω(n+k)	θ(n+k)	O(n^2)	O(n)
Radix Sort	Ω(nk)	θ(nk)	O(nk)	O(n+k)
Counting Sort	Ω(n+k)	θ(n+k)	O(n+k)	O(k)
Cubesort	Ω(n)	θ(n log(n))	O(n log(n))	O(n)

Problem Solving Patterns

1. **Pre-sorting to Simplify** - many array problems become trivial if the array is sorted first.
2. **Interval Merging (Sort by Start Time)** - when we want to combine overlapping intervals into one. If current.start < previous.end, they overlap.
3. **Interval Selection (Sort by End Time)** - when we want to maximize the number of non-overlapping items we can pick.
4. **Counting Sort** - when range is small (lowercase letters, ages), use a frequency array instead of std::sort.

Interview Questions

1. Why is QuickSort usually preferred over Merge Sort for arrays?
2. What is the worst-case time complexity of QuickSort? How do we avoid it?
3. Implement Quicksort
4. Implement Mergesort
5. Merge Sorted Array
6. Squares of a Sorted Array
7. Sort Colors
8. Merge Intervals
9. Non-overlapping Intervals

Commented [VJ6]: *QuickSort is in-place O(log N) stack space. Merge Sort requires O(N) extra memory for the temporary arrays.*
Merge Sort is preferred for Linked Lists because lists don't offer random access (bad for QuickSort's pivot swapping), and Merge Sort doesn't require extra space for lists (just pointer manipulation).

Commented [VJ7]: *O(n^2), it happens if the pivot chosen is always the smallest or largest element (sorting an already sorted array). To fix it use randomized pivot.*

10. Minimum Number of Arrows to Burst Balloons
11. Meeting Rooms II
12. Largest Number
13. Sort Characters By Frequency
14. Sort List
15. Insertion Sort List
16. Car Fleet
17. (★★★) Maximum Gap

Search Algorithms

Basics

Searching is the algorithmic process of finding a particular item in a collection of data.

Linear Search - iterate through the array from start to end. Return index if found, else -1. This is the only option for unsorted data.

A function that is either entirely non-increasing or non-decreasing is **monotonic** and can be Binary Searched with O(log n) time.

Algorithms

Binary Search - divide the search space in half. Compare target with the mid element. If target < mid, search the left half, else search the right half.
Ternary Search - similar to Binary Search but divides the array into three parts using two midpoints (m1, m2). Used to find the peak/valley of a function that increases then decreases.
Exponential Search - find range where element is present by checking indices 1,2,4,8,16..., then perform Binary Search in that range. Useful for unbounded/infinite streams.
Interpolation Search - an improvement on Binary Search for uniformly distributed data. Instead of always picking the middle, it uses a formula to guess the position.

Problem Solving Patterns

1. **Binary Search on Answer** - binary search over the range of possible answers. Use when the problem asks to "Minimize the Maximum..." or "Find the smallest X such that...". The key insight is monotonicity: if a value k works, does k+1 also work?
2. **Rotated Sorted Array** - one half of the array will always be sorted. Identify which half is sorted. If the left side is sorted, check if the target lies within that range. If it does, eliminate the right side, otherwise, the target must be in the unsorted right side.

3. **2D Matrix Reduction** - searching in a 2D matrix depends heavily on how the data is sorted.
   3a. Standard Reduction - if the matrix is sorted row-by-row, treat the entire matrix as a single flattened array. A index mid maps to the matrix at row mid / cols and column mid % cols.
   3b. Staircase Reduction - if rows and columns are sorted independently, start at the top-right corner, moving left decreases the value, and moving down increases the value. We can discard an entire row or column in each step.

Interview Questions

1. Binary Search
2. Search Insert Position
3. Sqrt(x)
4. Find First and Last Position of Element in Sorted Array
5. Find Peak Element
6. Search in Rotated Sorted Array
7. Find Minimum in Rotated Sorted Array
8. Search in Rotated Sorted Array II
9. Search a 2D Matrix
10. Search a 2D Matrix II
11. Koko Eating Bananas
12. Capacity To Ship Packages Within D Days
13. Minimum Number of Days to Make m Bouquets
14. Aggressive Cows
15. Search in a Sorted Array of Unknown Size
16. (**) Maximum Running Time of N Computers
17. (*) Split Array Largest Sum
18. (***) Median of Two Sorted Arrays

Two Pointers

Basics

Two Pointers technique involves using two distinct indices to traverse a data structure (usually an array or string) to save time and space. It is generally used to reduce complexity from O(n^2) to O(n).

Types of Two Pointers:

1. **Converging (Opposite Ends)-** one pointer starts at the beginning (left = 0), the other at the end (right = n-1). They move toward the center.
2. **Fast & Slow (Same Direction)**- both start at the beginning, but one moves faster.
3. **Merging (Independent)**- two pointers tracking position in two separate arrays.

Problem Solving Patterns

1. **The Converging Pointers** - one pointer at the start (left) and one at the end (right). In every step, evaluate the sum of the elements at them. If the sum is greater than target, move the right pointer to the left to decrease sum. If the sum is smaller than the target, move the left pointer to the right.
2. **Writer-Reader Logic** - two pointers moving in the same direction. The read pointer scans every element of the array. The write pointer tracks the position where the next valid element should be placed. If nums[read] meets the condition, copy it to nums[write] and increment write. If it doesn't meet the condition, the read pointer skips it.
3. **Dutch National Flag** - used to categorize elements into three distinct groups. It requires three pointers. low tracks the boundary of the first group, high tracks the boundary of the last group, and mid is the current element being examined. If nums[mid] belongs to the low group, swap it with nums[low] and advance both. If it belongs to the high group, swap it with nums[high] and decrement high.
4. **Fixed Point + Two Pointers (The N-Sum Approach)** - used to find three or more elements that sum to a target.

Interview Questions

1. Valid Palindrome
2. Remove Duplicates from Sorted Array
3. Move Zeroes
4. Remove Element
5. Is Subsequence
6. Remove Duplicates from Sorted Array II
7. Two Sum II- Input Array Is Sorted
8. 3Sum
9. 3Sum Closest
10. 4Sum
11. String Compression
12. Container With Most Water
13. Reverse Words in a String
14. (*) Trapping Rain Water

Sliding Window

Basics

Sliding Window is an optimization technique used primarily on arrays and strings. It converts nested loops into a single loop by maintaining a window of elements that satisfies certain constraints.

Problem Solving Patterns

1. **The Static Window** - initialize the window with the first k elements, then iterate through the rest of the array. In each step, add the new element entering the window (index i) and subtract the element leaving the window (index i-k).
2. **The Dynamic Window (Shrinkable)** - move right pointer to add elements until the window becomes valid. Once valid, try to make it smaller by moving left pointer. Keep shrinking while the window remains valid.
3. **The Dynamic Window (Expandable)** - move right pointer to add elements until the window becomes invalid, then move left pointer to fix it.
4. **Exact Count via At Most Subtraction** - used when the problem asks for an exact constraint. Standard sliding windows track "at most" or "at least" easily, but struggle with exact matches. Calculate countAtMost(K) - countAtMost(K- 1).

Traps & Anti-Patterns

1. **The Negative Number Trap** - expanding the right pointer must always increase or maintain the sum, and shrinking the left pointer must always decrease it. If the array contains negative numbers, expanding the window might decrease the sum (we no longer know which pointer to move to reach the target).
2. **The Subsequence vs Subarray** - sliding windows only work on contiguous blocks. If the problem allows us to skip or drop elements from the middle of the sequence, the sliding window completely falls apart.
3. **The Non-Monotonic Condition** - sliding window requires monotonicity. We must have strict, deterministic rules for shrinking and expanding. If removing an element from the left can either help or hurt target condition unpredictably, the monotonic property is destroyed.

Interview Questions

1. Maximum Average Subarray I
2. Find All Anagrams in a String
3. Permutation in String

4. Longest Substring Without Repeating Characters
5. Longest Repeating Character Replacement
6. Max Consecutive Ones III
7. Fruit Into Baskets
8. Longest Subarray of 1's After Deleting One Element
9. Minimum Size Subarray Sum
10. Subarray Product Less Than K
11. Count Number of Nice Subarrays
12. Binary Subarrays With Sum
13. (⋆) Minimum Window Substring
14. (⋆⋆⋆) Maximum Fruits Harvested After at Most K Steps

Graphs - Traversal & Connectivity

Basics

Graph is a non-linear data structure consisting of nodes (vertices) and edges (connections).

Representation:

1. **Adjacency List [O(V + E)]**: An array of lists (Map<int, List<int>>). Index i contains a list of nodes connected to i. Most common. Best for sparse graphs.
2. **Adjacency Matrix [O(V^2)]**: A 2D array grid[i][j] where 1 means connection. Use for dense graphs or when edge lookup needs O(1).
3. **Implicit Graph (The Grid):** A 2D Matrix where cells are nodes, and neighbors (Up, Down, Left, Right) are edges.

Usage

- **Social Networks**- finding friends of friends (BFS), suggesting connections.
- **GPS Navigation**- representing intersections as nodes and roads as edges.
- **Web Crawlers**- visiting links on a page (DFS/BFS) to index the internet.

Problem Solving Patterns

1. **Flood Fill (Islands Variation)** - used to identify connected components. Iterate through the graph and upon finding an unvisited node, launch BFS/DFS to visit all connected nodes and mark them as visited (or sink them by changing 1 to 0).
2. **Level-Order Shortest Path** - used in unweighted graphs. Use BFS to explore neighbors layer-by-layer. The first time we reach a target node, it is guaranteed to be the shortest path.

3. **Multi-Source BFS** - used when we have multiple starting points. Push all starting nodes into the Queue at t=0. The distance to any empty cell is determined by the closest source.
4. **Graph Coloring (Bipartite Check)** - used to split a graph into two independent sets. Perform BFS/DFS and assign the first node Color A. Assign all neighbors Color B. If we encounter a neighbor that already has Color A, the graph is not bipartite.
5. **Cycle Detection (DFS Recursion Stack)** - used in directed graphs. Maintain a visited set (global) and a recursion_stack set (current path). If we encounter a node currently in the recursion_stack, a cycle exists.

Interview Questions

1. Number of Islands
2. Max Area of Island
3. Rotting Oranges
4. Walls and Gates
5. Pacific Atlantic Water Flow
6. Surrounded Regions
7. Clone Graph
8. Number of Connected Components in an Undirected Graph
9. Is Graph Bipartite?
10. Possible Bipartition
11. 01 Matrix
12. Graph Valid Tree
13. (★) Word Ladder
14. (★) Making A Large Island
15. (★) Swim in Rising Water
16. (★★) Bus Routes

Graphs - Optimization & Structures

Basics

Graph Optimization algorithms solve specific constraints: finding the best path, resolving dependencies, or managing dynamic connections.

Algorithms

1. **Topological Sort (Kahn's Algorithm)** [O(V+E)]
   Order nodes in a Directed Acyclic Graph (DAG) such that for every edge A-> B, A comes before B.

1) Calculate **In-Degree** for all nodes.
2) Push all nodes with In-Degree == 0 into a Queue.
3) For each neighbor, decrement its In-Degree. If it becomes 0, push to Queue.

2. **Union-Find (Disjoint Set Union- DSU) [O(1) Amortized]**
   Efficiently group elements into sets and check if two elements belong to the same set.
   - Find(x): Returns the representative (root) of x's set. (uses **Path Compression** to make future lookups O(1).
   - Union(x, y): Merges the sets containing x and y. (uses **Union by Rank** to keep tree flat).
3. **Dijkstra's Algorithm [O(E logV)]**
   Gives the shortest path in a weighted graph (non-negative weights). Uses standard BFS, but use a **Min-Priority Queue** instead of a regular Queue.
4. **Bellman-Ford [O(VE)]**
   Computes shortest paths with negative weights. Can detect negative cycles. Slower than Dijkstra. Works by relaxing all edges V-1 times to find the shortest paths. To check for negative cycles, relax all edges one more time.
5. **Floyd-Warshall Algorithm (All-Pairs Shortest Path) [$O(V^3)$]**
   1) Initialize a matrix dist where dist[i][j] = edge weight, and dist[i][i] = 0. Set unconnected nodes to infinity.
   2) Use DP with three nested loops. The outer loop k represents the intermediate node allowed in the path.
   3) For every pair (i, j), check if going through k is shorter than the current direct path: dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]).
6. **Kruskal's Algorithm: (Greedy + Union-Find) [O(E logE)]**
   4) Sort all edges by weight.
   5) Iterate through edges. Use Union-Find to check if the two nodes are already connected.
   6) If not connected, add edge to MST and Union them.
7. **Prim's Algorithm: (Greedy + Priority Queue) [O(E logV)]**
   1) Start from an arbitrary node. Push all its edges to a Min-Heap.
   2) Pop the smallest edge. If it leads to an unvisited node, add to MST and push that node's edges to Heap.
8. **Hierholzer's Algorithm (Eulerian Path/Circuit) [O(E)]**
   Circuit Requirements - every node must have an **even degree** (undirected) or **indeg == outdeg** (directed).
   Path Requirements - exactly 2 nodes have odd degree (undirected) or Start has **Out = In + 1** and End has **In = Out + 1** (directed).
   1) Start DFS from a valid node.

2) Traverse edges and remove them as we go, to ensure they are used only once.
3) Add the current node to the result list only after visiting all its neighbors (post-order).
4) Reverse the result list to get the path.

Problem Solving Patterns

1. **Dependency Resolution (Topological Sort)** - use when finding a valid order for items with dependencies. If the graph contains a cycle, a topological sort is not possible.
2. **Dynamic Connectivity (Union-Find)** - use to quickly group elements or count connected components without traversing the graph every time. Initialize every node as its own parent. For every connection, union the sets and decrement the total component count. Use find to check if two nodes are already connected.
3. **Dijkstra with State** - use when finding a shortest path with extra constraints. A standard visited[node] array fails because we might revisit a node with a better state. Instead, track visited[node][state] and store {cost, node, state} in the priority queue.
4. **Connect All Points (Kruskal's MST)** - calculate distances between all pairs to form edges, sort them by weight, and iterate. Use Union-Find to add edges only if the two nodes are not already connected.
5. **Small Graph All-Pairs (Floyd-Warshall)** - use only when the graph is small (N ≤ 500) and we need distances between every pair.

Interview Questions

1. Course Schedule
2. Course Schedule II
3. Parallel Courses
4. Redundant Connection
5. Network Delay Time
6. Cheapest Flights Within K Stops
7. Path With Maximum Probability
8. Path With Minimum Effort
9. Find the City With the Smallest Number of Neighbors at a Threshold Distance
10. Min Cost to Connect All Points
11. (★★) Accounts Merge
12. (★★) Alien Dictionary
13. (★★) Number of Islands II
14. (★★★) Minimize Malware Spread
15. (★★★) Shortest Path Visiting All Nodes
16. (★★) Reconstruct Itinerary

Greedy Algorithms

Basics

Greedy Algorithms build up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit. The hope is that by choosing the locally optimal move at each step, we end up with the globally optimal solution.

Note: Unlike DP, a Greedy never reconsiders its choices. Once a decision is made, it is final.

Problem Solving Patterns

1. **Resource Assignment (Sort Both Arrays)**- used when matching two sets of data to satisfy constraints.
2. **The Max Reach** - used when determining if we can reach the end of an array or minimum steps to do so. Maintain a variable max_reach. Iterate through the array. At every step i, update `max_reach = max(max_reach, i + nums[i])`. If i ever exceeds max_reach, we are stuck.
3. **Merge Cost Minimization (Huffman Pattern)** - used when the cost of combining two items equals their sum, and we want to minimize the total cost of combining everything. Always combine the two smallest available items first. Use a Min-Heap.

Interview Questions

1. Assign Cookies
2. Lemonade Change
3. Array Partition I
4. Maximum Units on a Truck
5. Best Time to Buy and Sell Stock
6. Boats to Save People
7. Jump Game
8. Jump Game II
9. Gas Station
10. Partition Labels
11. Two City Scheduling
12. Remove K Digits
13. Advantage Shuffle
14. (★) Candy
15. (★★) Patching Array
16. (★★★) Minimum Number of K Consecutive Bit Flips
17. (★★) Minimum Number of Taps to Open to Water a Garden

Bitwise Manipulation

Basics

Bitwise Operations work directly on the binary representation of numbers. They are extremely fast and are often used to optimize space or perform specific mathematical tricks.

- **&** - 1 if both bits are 1 (intersection).
- **|** - 1 if at least one bit is 1 (union).
- **^** - 1 if bits are different (difference).
- **~** - inverts all bits.
- **<<** - shifts bits left, filling with 0. Equivalent to **multiplying by 2**.
- **>> -** shifts bits right. Equivalent to **int division by 2**.

Computers store negative numbers using Two's Complement. To get-x, invert all bits of x and add 1.

$$-n = (\sim n) + 1$$

Usage

- **Flags/Permissions** - storing multiple boolean states in a single integer.
- **Sets** - representing a small set of numbers (0-31) using a single integer (**Bitmasking**).
- **Compression** - packing data into smaller types.
- **Fenwick Trees** - using **n &-n** to traverse the tree.

Problem Solving Patterns

1. **Bit Management**
    a. Check if k-th bit is set: `(n >> k) & 1`
    b. Set k-th bit: `n | (1 << k)`
    c. Clear k-th bit: `n & ~(1 << k)`
    d. Toggle k-th bit: `n ^ (1 << k)`
    e. Set last k bits: `n | ~(~0 << k)`
    f. Clear last k bits: `n & ~0 << k`
2. **Brian Kernighan's Algorithm (Strip Last Set Bit)** - used to count the number of 1s or check for powers of 2 or to remove right most set bit. Formula: `n & (n- 1)`.
   Subtracting 1 from a number flips all bits after the rightmost '1'. Performing AND with the original removes that rightmost '1'.
3. **Isolation (The Lowbit / LSOne)** - used to isolate the rightmost '1' bit and set all others to 0. Essential for Fenwick Trees. Formula: `n & (-n)`.

Since -x is (~x) + 1, the carry from adding 1 ripples until it hits the first zero (which was originally the first one). The AND operation filters out everything else.

4. **The XOR (The Cancel Out)** - if we XOR a list of numbers where every number appears twice except one, the result is that single number.
5. **Bitmask Iteration** - used to generate all possible subsets of a set (Backtracking/DP). If we have N elements, iterate from 0 to 2^N- 1. For each number mask, iterate j from 0 to N-1. If (mask >> j) & 1 is true, include the j-th element in the current subset.

Interview Questions

1. Single Number
2. Number of 1 Bits
3. Counting Bits
4. Reverse Bits
5. Power of Two
6. Missing Number
7. Single Number II
8. Bitwise AND of Numbers Range
9. Maximum Product of Word Lengths
10. (**) Maximum XOR of Two Numbers in an Array

Divide and Conquer

Basics

Divide and Conquer is a paradigm that breaks a problem into smaller, disjoint subproblems, solves them recursively, and then combines their results to solve the original problem.

1. **Divide**: Split the problem into subproblems of smaller size (usually N/2).
2. **Conquer**: Solve the subproblems recursively. If the problem is small enough solve it directly.
3. **Combine**: Merge the solutions of the subproblems to form the solution for the original problem.

Master Theorem

When we write a Divide & Conquer, the time complexity is not obvious. Used to analyze time complexity. If $T(n) = aT(n/b) + O(n^d)$:

a) If $a > b^d$ - work grows faster than problem shrinks ($O(n^{\log_b a})$)
b) If $a = b^d$ - work equals shrink rate ($O(n^d \log n)$)

c) If $a < b^d$ - work shrinks faster ($O(n^d)$)

a	How many recursive calls do we make?
b	How much do we divide the input?
d	How much work do we do outside the recursion?

Example: Split the array into 3 parts and solve recursively each of that parts, then merge in linear time.

Solution: a = 3 calls, b = 3 splits, d = 1 (merge is linear). So $3 = 3^1$, it is case 2 $a = b^d$. Time complexity is O(n logn).

Usage

- **Sorting** - Merge Sort, Quick Sort.
- **Large Number Math** - Karatsuba Algorithm, Fast Exponentiation.
- **Parallelism** - MapReduce is essentially distributed Divide & Conquer.

Problem Solving Patterns

1. **Modified Merge Sort (Counting Inversions)** - used when the problem asks about "pairs that satisfy a condition". Perform Merge Sort but change merge part. When merge two sorted subarrays, if we pick an element from the right array to place in the sorted buffer before an element from the left array, it implies that the right element is smaller than all remaining elements in the Left array. Add left_len - left_ptr to total count.
2. **Fast Exponentiation** - used to calculate x^n in O(log n).
    - If n is even: $x^n = x^{n/2} * x^{n/2}$
    - If n is od: $x^n = x* x^{n/2} * x^{n/2}$
3. **Divide and Scale (Radix-2 / Parity Splitting)** - used when a problem asks to construct a sequence or evaluate an array where elements have strict arithmetic relationships. Split the array by parity (even/odd indices or values) to break the arithmetic relationship between the two halves.

Interview Questions

1. Majority Element
2. Pow(x, n)
3. Beautiful Array
4. (*) Longest Substring with At Least K Repeating Characters
5. (**) Different Ways to Add Parentheses
6. (**) Count of Smaller Numbers After Self
7. (**) Reverse Pairs

Backtracking

Basics

Backtracking is an algorithmic paradigm that tries to solve problems recursively by building a solution incrementally, one piece at a time, removing those solutions that fail to satisfy the constraints of the problem at any point of time. It is essentially DFS on a state-space tree.

Choice: What choice do we make at this step?
Constraint: Does this choice satisfy the problem conditions?
Goal: Have we reached a valid solution?
Backtrack: If the choice doesn't lead to a solution or we need to find all solutions, we undo the choice (reset state) and try the next option.

Usage

- **Combinatorial Problems** - Generating all permutations, subsets, or combinations.
- **Constraint Satisfaction Problems** - Sudoku, N-Queens, Crosswords.

Problem Solving Patterns

1. **The "Include vs Exclude" (Subsets Pattern) [$O(2^n)$]** - used when generating subsets or solving knapsack-like variations. At every index, we have two choices: either include the element in the current subset or skip it.
2. **The N-ary Tree Loop** - used when the branching factor is variable. Iterate through the input array starting from a specific index.
    - **Combinations [$O(n\ 2^n)$]** (order doesn't matter): Pass a startIdx to the recursive function. The loop runs from `i = startIdx to n`. This prevents reusing previous elements.
    - **Permutations [$O(n\ n!)$]** (order matters): The loop always runs from 0 to n. We need a `visited[]` array or a swap mechanism to ensure we don't use the exact same element index twice in the same branch.
    - **With Duplicates**: If the input array contains duplicates and we want unique results, Sort the array first. Check: `if (i > start && nums[i] == nums[i-1]) continue;`
3. **Grid Backtracking (DFS + State)** - mark the current cell as visited. Explore all directions. To backtrack, unmark the cell after returning from recursion so other paths can use it.
4. **Backtracking with Memoization** - if the state can be condensed, and we are recalculating the same state multiple times, cache the result.

Interview Questions

1. Subsets

2. Subsets II
3. Permutations
4. Permutations II
5. Combinations
6. Combinations Sum
7. Combination Sum II
8. Combination Sum III
9. Letter Combinations of a Phone Number
10. Palindrome Partitioning
11. Restore IP Addresses
12. Generate Parentheses
13. Letter Case Permutation
14. Word Search
15. (★) N-Queens
16. (★) N-Queens II
17. (★) Sudoku Solver
18. (★) Unique Paths III
19. (★★★) Expression Add Operators
20. (★★★) Word Break II

Dynamic Programming - 1D Arrays & State Machines

Basics

Dynamic Programming is recursion with caching. If a problem has **Overlapping Subproblems** (solve the same sub-problem multiple times) and **Optimal Substructure** (optimal solution can be constructed from optimal solutions of its sub-problems), it is DP.

- Top-Down (Memoization) - Write the recursive function solve(n). Check if memo[n] exists, if yes, return it. If no, calculate, cache it, and return.

  It is easy to write (follows the recurrence naturally), but cannot easily optimize space (recursion stack).

- Bottom-Up (Tabulation) - Initialize a dp array. Fill it iteratively starting from the base case dp[0] up to the target dp[n].

  It allows space optimization (reducing dp array to variables), but we must solve all subproblems even if some aren't needed.

Problem Solving Strategy

1. **Define State** - What variables define our position?
2. **Derive Recurrence Formula**
3. **Implement Top-Down first** (if struggling).
4. **Refactor to Bottom-Up** for the final solution to show off space optimization.

Note: Most 1D DP problems depend only on the previous 1 or 2 states. In Bottom-Up, we can replace the array with variables (prev, curr) to achieve O(1) space.

When we are trying to figure out what variables belong in your memoization table memo[][]:

1. Does this variable limit my future choices? **KEEP**
2. Does this variable just tell me how well I did in the past? **THROW**

Problem Solving Patterns

1. **Linear Sequence** - problems where the answer at i depends on i-1 or i-2.
2. **Longest Increasing Subsequence (LIS)** - dp[i] = length of LIS ending at index i ($O(n^2)$).
   We can do in O(n logn). Maintain a list tails. tails[i] stores the smallest tail of all increasing subsequences of length i+1. Iterate through nums. For each x, If x is larger than all tails, append it. Else, use Binary Search to find the smallest element in tails >= x and replace it.
3. **Kadane's Algorithm** - used to find the contiguous subarray with the largest sum.
   Is the number itself bigger than the number + previous sum? If yes, the previous sum was dragging us down, so cut it off.
   Recurrence: dp[i] = max(nums[i], nums[i] + dp[i-1])
4. **State Machine DP** - when the decision depends on a state. Use multiple DP arrays or variables representing states.

Interview Questions

1. Climbing Stairs
2. Min Cost Climbing Stairs
3. Decode Ways
4. House Robber
5. House Robber II
6. Maximum Subarray
7. Maximum Sum Circular Subarray
8. Maximum Product Subarray
9. Longest Increasing Subsequence
10. Number of Longest Increasing Subsequence
11. Largest Divisible Subset

12. Paint Fence
13. Paint House
14. Best Time to Buy and Sell Stock with Transaction Fee
15. (**) Paint House II
16. (**) Russian Doll Envelopes
17. (**) Best Time to Buy and Sell Stock with Cooldown
18. (***) Best Time to Buy and Sell Stock III
19. (***) Best Time to Buy and Sell Stock IV

Dynamic Programming - Knapsack & Subsets

Basics

Knapsack Problems involve a set of items, each with a weight and a value, and a container with a maximum capacity. The goal is to maximize the total value without exceeding the capacity.

For every item i and capacity w, we have two choices:

1) **Include**: Add the item's value, but reduce capacity (**value[i] + dp[i - 1][w - weight[i]]**).
2) **Exclude**: The value remains the same as without this item (**dp[i - 1][w]**).

Optimization: A 2D dp[i][w] array uses O(NW) space. Since dp[i] only depends on dp[i-1], we can optimize to a 1D array dp[w] of size O(W).

Problem Solving Patterns

1. **0/1 Knapsack (Finite Items)** - each item can be used at most once.
   Recurrence: **dp[w] = max(dp[w], dp[w - weight] + value)**
   When optimizing to 1D, the inner loop must iterate BACKWARDS (from W down to weight). If we iterate forward, dp[w] would use the value of dp[w-weight] from the current iteration, effectively using the item multiple times. Iterating backwards ensures we use values from the previous iteration (without the current item).
2. **Unbounded Knapsack (Infinite Items) -** each item can be used unlimited times.
   Recurrence: dp[w] = max(dp[w], dp[w- weight] + value)
   The inner loop must iterate FORWARDS (from weight up to W). Since we have infinite supply, we want to use the updated dp[w-weight] (which might already include the current item) to stack another instance of the same item.
3. **Combinations vs Permutations (Coin Change)**
   - **Combinations** (order doesn't matter) - outer loop -> coins, inner loop -> amount. We process coin 1 for all amounts, then coin 2. This forces the order 1...1, 2...2.

- **Permutations** (order matters) - outer loop -> amount, inner loop -> coins. For amount 5, we try ending with coin 1, then ending with coin 2. This builds all sequences.

4. **The Target Sum Transformation** - problems asking to assign + and- signs to reach a target. Let P be the sum of positive numbers, N be the sum of negative numbers:
   - P - N = Target
   - P + N = TotalSum

   Adding them: 2P = Target + TotalSum => P = (Target + TotalSum) / 2

   Find a subset of numbers that sums to (Target + TotalSum) / 2. This is 0/1 Knapsack.

Interview Questions

1. Partition Equal Subset Sum
2. Ones and Zeroes
3. Coin Change
4. Coin Change II
5. Combination Sum IV
6. Perfect Squares
7. Integer Break
8. (★) Target Sum
9. (★★) Last Stone Weight II
10. (★★★) Partition to K Equal Sum Subsets
11. (★★★) Tallest Billboard

Dynamic Programming - 2D Grids & Paths

Basics

These problems involve moving through a gird to reach a target or accumulate a value. Movement is usually restricted to Right and Down. If movement is allowed in 4 directions, it is graph problem, not DP (cycles exist). DP only works on DAGs.

Optimization: Standard grid DP takes O(MN) space. Since dp[r][c] usually only depends on the current row r and the previous row r-1, we can optimize to O(n) space using two rows.

Problem Solving Patterns

1. **Path Counting & Sums (Top-Left to Bottom-Right)**
   Recurrence: dp[r][c] = grid[r][c] + min(dp[r-1][c], dp[r][c-1])

2. **Health/Survival (Bottom-Right to Top-Left)** - used when the decision depends on the future state, not the past.
   Recurrence: dp[r][c] = max(1, min(dp[r][c + 1], dp[r + 1][c]) - grid[r][c])
3. **Geometric DP (Squares)** - finding the largest square of 1s in a binary matrix.
   Recurrence: dp[r][c] = min(dp[r-1][c], dp[r][c-1], dp[r-1][c-1]) + 1
4. **Multiple Agents** - involving two people moving simultaneously. We cannot run DP twice because the first path changes the grid state for the second path. We must simulate them moving together.
   State: dp[step][r1][r2]
   Note: We don't need c1 and c2 because step = r1 + c1 = r2 + c2.
5. **Tower Pattern** - used when moving down a staggered/triangular grid where each cell connects to exactly two adjacent cells in the row below it. It is usually best to process this Bottom-Up starting from the last row to avoid out-of-bounds checks and simplify the state.
   Recurrence: dp[r][c] = triangle[r][c] + min(dp[r+1][c], dp[r+1][c+1])
6. **DP + Monotonic Stack** - used for finding the maximum rectangle area in a 2D matrix. Treat each row of the grid as the base of a histogram. Use DP to accumulate the height of consecutive 1s column by column, then apply the Monotonic Stack algorithm to find the max area for that row.

Interview Questions

1. Unique Paths
2. Unique Paths II
3. Minimum Path Sum
4. Triangle
5. Minimum Falling Path Sum
6. Maximal Square
7. Champagne Tower
8. (★) Longest Increasing Path in a Matrix
9. (★★★) Dungeon Game
10. (★★★) Count Submatrices with All Ones
11. (★★) Cherry Pickup
12. (★★) Cherry Pickup II

Dynamic Programming - Intervals & Strings

Basics

These problems involve finding an optimal structure within a sequence or comparing two sequences. The subproblems are defined by Start and End indices. Time complexity is usually $O(N^2)$ or $O(N^3)$.

Optimization: Many string DPs only need the previous row dp[i-1], allowing O(N) space. Interval DPs generally require full $O(N^2)$ tables.

Problem Solving Patterns

1. **Longest Common Subsequence (LCS)** - given two strings s1 and s2, finding the longest subsequence present in both.
   State: dp[i][j] = LCS of s1[0...i] and s2[0...j]
   Recurrence:
   1) **Match:** If s1[i] == s2[j]
      dp[i][j] = 1 + dp[i-1][j-1] (extend the match found at previous indices).
   2) **No Match:** If s1[i] != s2[j]
      dp[i][j] = max(dp[i-1][j], dp[i][j-1]) (skip character from s1 or skip character from s2).

   Variation - instead of just skipping, we have Insert, Delete, or Replace operations.

   dp[i][j] = min(Insert, Delete, Replace)
   Insert = 1 + dp[i][j-1], Delete = 1 + dp[i-1][j], Replace = 1 + dp[i-1][j-1]

2. **Palindromic Substrings** - finding palindromes within a string.
   State: dp[i][j] is true if s[i...j] is a palindrome
   Recurrence:
   1) Length 1: always true (dp[i][i] = true)
   2) Length 2: True if s[i] == s[i+1]
   3) Otherwise: dp[i][j] = (s[i] == s[j]) && dp[i+1][j-1] (ends match and the inner substring is already a palindrome)

   Optimization: Iterate through centers and expand left and right. $O(N^2)$ time but O(1) space.
3. **Matrix Chain Multiplication (MCM)** - we want to find the optimal way to combine or split the range i to j.
   State: dp[i][j] = max value obtainable from subarray nums[i...j]
   Recurrence: dp[i][j] = max(dp[i][k] + dp[k+1][j] + cost_of_split)

Iterate through all possible split points k between i and j. The order of iteration must be by length (len 2, then 3, then 4...). Solve small ranges first to build larger ranges.

4. **Minimax + DP** - used when two players take turns making optimal moves from the ends of an array. The score is calculated as the current player's choice minus the opponent's optimal future score.
   State: dp[i][j] = the max score the current player can achieve from nums[i...j]
   Recurrence: dp[i][j] = max(nums[i] - dp[i+1][j], nums[j]- dp[i][j-1])

Interview Questions

1. Longest Common Subsequence
2. Longest Common Substring
3. Edit Distance
4. Delete Operation for Two Strings
5. Interleaving String
6. Longest Palindromic Substring
7. Palindromic Substrings
8. Longest Palindromic Subsequence
9. Word Break
10. Stone Game
11. (★★) Stone Game III
12. (★★) Distinct Subsequences
13. (★★★) Shortest Common Supersequence
14. (★★★) Palindrome Partitioning II
15. (★★★) Burst Balloons
16. (★★★) Minimum Cost to Merge Stones
17. (★★) Regular Expression Matching
18. (★★) Wildcard Matching

String Matching Algorithms

Basics

String Matching involves finding occurrences of a Pattern (length M) within a Text (length N).

- **Naive Approach**: Check every starting index. **T = O(NM).**
- **Optimal Approach**: Use preprocessing to avoid redundant comparisons. **T = O(N+M).**

Algorithms

1. **Rabin-Karp (Rolling Hash)** - when we need to match a pattern of fixed length, or when doing multiple pattern matching. It uses a Sliding Window combined with a hash. Compute a hash for the pattern. Then compute a hash for the first M characters of the text. Slide the window one character at a time. Subtract the outgoing character and add the incoming character.
   The Math (Base B, Modulo MOD):
   Usually B = 256 (or prime like 31) and MOD = $10^9 + 7$.

Remove outgoing char (Left)	Hash = Hash - (text[left] * B^{M-1}) (mod MOD)
Shift left (Multiply by Base)	Hash = Hash * B (mod MOD)
Add incoming char (Right)	Hash = (Hash + text[right]) (mod MOD)

` *Note: Due to hash collisions if $Hash_{window} == Hash_{pattern}$, we must do an O(M) string comparison to guarantee it's an exact match.*

2. **KMP Algorithm (Knuth-Morris-Pratt)** - avoids backtracking in the text by knowing exactly how much of the pattern has already been matched.
   The LPS Array (Longest Prefix Suffix):
   Before searching, preprocess the pattern to create an lps array of size M.
   lps[i] = the length of the longest proper prefix of pattern[0...i] that is also a suffix of pattern[0...i].
   Maintain two pointers: i for text, j for pattern.
   - If text[i] == pattern[j], increment both.
   - If they mismatch and j > 0, do not reset i. Set **j = lps[j- 1]** (the previous characters matched, so we shift the pattern to align the longest known prefix-suffix).
   - If they mismatch and j == 0, increment i.

Problem Solving Patterns

1. **Dual-Directional Hashing (Prefix & Suffix)** - used to find palindromes or happy prefixes without $O(N^2)$.
   - **Forward Hash**: Shift the accumulated hash left and add the new character.
   - **Backward Hash**: Multiply the new character by the current base power and add it to the accumulated hash.
2. **Binary Search + Rabin-Karp** - for a given length mid, slide a window of length mid across the text and store the hashes in a Set.
3. **Double Hashing (Anti-Collision Strategy)** - run the rolling hash math using two different prime modulos simultaneously. Pack the two 32-bit hashes into a single 64-bit integer using (hash1 << 32) | hash2.

Interview Questions

1. Find the Index of the First Occurrence in a String
2. Repeated Substring Pattern
3. Rotate String
4. (★) Repeated String Match
5. (★) Shortest Palindrome
6. (★) Longest Happy Prefix
7. (★★) Longest Duplicate Substring

Segment Trees

Basics

Segment Tree is a binary tree used for storing information about intervals or segments. It allows answering range queries (Sum, Min, Max, GCD) and updating elements efficiently.

A Prefix Sum array answers range sum queries in O(1) but takes O(N) to update an element. A Segment Tree balances this, making both updates and queries O(log N).

- **Leaves**: Represent individual elements of the original array.
- **Internal Nodes**: Represent the merged result (sum, min, max) of its left and right children.
- **Indices**: For a node at index i, its left child is 2i + 1, its right child is 2i + 2, and its parent is (i - 1) / 2.

Core Mechanics

1. **Build** - recursively divide the array in half until start == end. On the way back up the recursion tree, merge the left and right children to build the internal nodes.
2. **Query** - given a target range [L, R] and the current node's range [start, end]:
    - **Total Overlap (L <= start and end <= R):** Return the current node's value.
    - **No Overlap (end < L or start > R):** Return a neutral value (0 for sum, INF for min).
    - **Partial Overlap:** Recursively query both left and right children and merge the results.
3. **Update** - traverse down to the leaf representing the updated index. Update the leaf, and on the way back up, recalculate the values of all its ancestors.

Problem Solving Patterns

1. **Point Update & Range Query (The Standard)** - used when elements in the array frequently change, and we need to repeatedly find the min/max/sum of a specific subarray.
2. **Lazy Propagation (Range Update)** - used when we need to update an entire range of elements at once. Instead of updating every leaf (O(N)), we store a lazy value at the internal node and stop traversing. We only push this lazy value down to its children later if we actually need to visit them. Keeps range updates at O(log N).
3. **Dynamic Segment Tree** - used when the range of possible values is massive but the actual number of elements is small. Instead of allocating a 4N array, we use actual Node objects with left and right pointers, instantiating children only when they are visited.
4. **Frequency / Order Statistic Tree** - instead of storing array values, the leaves represent the numbers themselves (from 0 to max_val), and the tree stores their frequencies. Used to quickly answer "how many numbers in the array are strictly smaller than X".

Interview Questions

1. Range Sum Query – Mutable
2. (★★) My Calendar I
3. (★★) My Calendar II
4. (★★) My Calendar III
5. (★★) Falling Squares
6. (★★★) Range Module

Geometric Algorithms

Basics

Geometry Algorithms typically involve points, lines, and shapes on a 2D Cartesian plane. The main challenge is translating mathematical formulas into code without falling into precision traps (like using double or float and failing equality checks).

- **Euclidean Distance:** $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
  Optimization: Never use sqrt() unless strictly required. Compare squared distance ($dx^2 + dy^2$) to keep everything as integers.
- **Manhattan Distance:** $|x_2 - x_1| + |y_2 - y_1|$
  Used when movement is restricted to a grid (up/down/left/right).
- **Slope**: $m = (y_2 - y_1)/(x_2 - x_1)$

Problem Solving Patterns

1. **The Hashable Slope (Collinearity)** - used to find how many points lie on the same line. Never use double for slopes due to precision errors. Instead, reduce the slope to its simplest fractional form using the Greatest Common Divisor. Store the pair (dy / gcd, dx / gcd) as a string or pair in a Hash Map.
2. **Rectangle Overlap / Intersection** - two 1D line segments [a1, a2] and [b1, b2] overlap if max(a1, b1) < min(a2, b2). To check if two 2D rectangles overlap, apply this 1D logic independently to both the X-axis and the Y-axis. The intersection area's width is min(RightXs)- max(LeftXs) and height is min(TopYs)- max(BottomYs).
3. **Cross Product (Orientation)**- used to determine if three points (p1, p2, p3) make a left turn, right turn, or are collinear without using division.
   Formula: (y2- y1) * (x3- x2) - (y3- y2) * (x2- x1). If the result is 0, the points are perfectly collinear.
4. **Sweep Line Algorithm -** let a vertical line sweep across the 2D plane from left to right. Break shapes into events (the left is an "add" event, the right is a "remove" event). Sort all events by their X-coordinate and process them in order, maintaining the active Y-coordinates in a multiset.
5. **Convex Hull (Monotone Chain)** - finding the smallest convex polygon that encloses a set of points. Sort points by X (then Y). Build the upper and lower halves of the hull using a Monotonic Stack, popping the top of the stack if the current point makes a non-left turn (using the Cross Product).

Interview Questions

1. Rectangle Overlap
2. Rectangle Area
3. Valid Square
4. Minimum Area Rectangle
5. (**) Max Points on a Line
6. (***) The Skyline Problem
7. (***) Perfect Rectangle
8. (***) Erect the Fence

Follow Up Questions

1. What if the input array is 500GB, but your machine only has 4GB of RAM?
   1. Break the data into 4GB chunks, sort them individually, write them to disk, and then use a Min-Heap (K-way merge) to merge them stream-by-stream back together.

2. MapReduce: Split the data across 1,000 machines. Map the data to key-value pairs, shuffle by key, and Reduce (aggregate) on separate nodes.
3. If the input is integers limited to a specific range, we can represent their presence using an array of bits rather than 32-bit integers, reducing memory by 32x.

2. What if the input never ends? It's a continuous stream of data.
    1. We can't sort an infinite stream. Use a Min-Heap of size K to constantly track the top K largest elements in O(log K) time per element.
    2. If we only care about the last 5 minutes of data, maintain a queue or a deque and aggressively evict old timestamps.
    3. If streaming characters and checking against a dictionary, advance pointers down a Trie node by node as characters arrive.
3. How would we run something in a multi-threaded environment?
    1. **Read-Heavy Workloads (Shared Locks):** Use a std::shared_mutex (Read-Write Lock). Allow infinite simultaneous readers, but exclusively lock out everyone when a writer is updating the data.
    2. **Lock Striping / Sharding**: Instead of locking the entire HashMap (which creates a massive bottleneck), lock individual buckets. This allows Thread A to write to Bucket X while Thread B writes to Bucket Y simultaneously.
    3. **Thread-Local Storage**: Have each thread compute its own local result, and then merge the local results at the very end to minimize locking.
    4. **Atomic Operations**: For simple counters, use std::atomic<int> (Compare-And-Swap) instead of expensive mutex locks.
4. We call get() 10 million times a second, but we only call add() once a month. How do you redesign this?
    1. **Read-Heavy**: Shift all the computational weight to the add() function. Pre-compute the answers, sort the arrays, and cache everything during the write phase so get() is strictly O(1).
    2. **Write-Heavy:** Buffer the writes in an append-only log or an unsorted list. Only do the heavy sorting/merging when someone actually calls get() (Lazy Evaluation).

Random

1. Maximum Sum BST in Binary Tree
2. Number of Provinces
3. Binary Search Tree to Greater Sum Tree
4. Insert Delete GetRandom O(1) - Duplicates Allowed
5. Maximum Amount of Money Robot Can Earn
6. Find the Maximum Number of Fruits Collected

7. Data Stream as Disjoint Intervals
8. Copy Linked List with Random Pointer
9. Valid Number
10. Palindrome Pairs